

Research on A Lightweight Service Mesh Architecture Using eBPF

Yukai Tan,
China Telecom
China, Beijing
tanyk6@chinatelecom.cn

Zeya Zhu
China Telecom
China, Beijing
zhuzy13@chinatelecom.cn

Shuo Quan
China Telecom
China, Beijing
quansh@chinatelecom.cn

Shen Gao
China Telecom
China, Beijing
gaos13@chinatelecom.cn

Abstract—Many applications deployed in the cloud are typically refactored as widgets in microservices, and these components run as containers in a Kubernetes environment. The containers then run on clusters of physical servers that interoperate with the data center network.

With this deployment, a large number of microservice containers running comes with resource consumption. The prevailing management architecture of microservices deepens this resource consumption on physical servers and increases network latency. We believe that this scenario could be further optimized.

In this paper, we argue for an improved service mesh architecture design (compared to the traditional one). It reduces the resource consumption and improves the performance of deployed microservice applications, while providing some end-to-end visibility to debug problems that occur in microservices. We present the design and provide preliminary implementation details using eBPF technology to illustrate the feasibility of the system.

Keywords-component: eBPF, service mesh, micro service

I. INTRODUCTION

As virtualized applications evolve towards microservices, it is important for cloud providers to be able to deploy efficient microservices applications on the cloud at a lower cost. Microservices technology can help developers manage applications more easily and can deploy and update applications faster. It can also help organizations scale applications more easily and can meet customer needs faster.

Cloud-native applications may consist of a large number of microservices, and a large number of microservice deployments suffer from three problems: 1) different programming language implementations 2) belonging to different tenants and 3) thousands of application instances. In this case, some microservices frameworks such as Spring Cloud, Dubbo, etc. implement various common semantic features needed for distributed system communication though: such as load balancing and service discovery. But it is actually very complex, such as Spring Cloud, aggregates many components. So in practice, the use of the above microservices framework will find a number of problems: invasive, high

upgrade costs, complex architecture, high learning costs, the inability to do programming language-independent and so on.

On this basis, a new architecture called service mesh, which can decouple from business and mainly solve the communication between microservices and microservices in complex network topology, has emerged. Its current form is generally a lightweight network agent and is deployed with applications in a side-car agent mode, while being transparent to business applications. service mesh technology is widely promoted for use as a configurable infrastructure for microservice applications.

There are two issues to be aware of when using service mesh technology. First, current service mesh architectures commonly use the side car model. The resource overhead (memory, CPU, etc.) of edge carts is a big challenge for edge cloud application deployment. The second is how to get end-to-end visibility in a complex service mesh architecture?

There are two main challenges in trying to optimize these two issues. Firstly, the service mesh implementation in the edge car model integrates several features, such as elastic links, traffic management, observability and transparency, into the edge car agent implementation. This also makes the agent consume huge resources when a large number of side car agents are running on the host. The second is that to achieve end-to-end observability in such a complex system requires tracing and data analysis of the host kernel network stack.

In this paper, we use the eBPF technique to implement the core functionality of the edge carriage down to the host kernel implementation and eliminate the edge carriage proxy mechanism. Compared with existing techniques, this implementation enables the edge carriage function to be implemented in the kernel with less overhead, and also reduces the packet path for packets to be forwarded between proxies and proxies for the purpose of reducing latency. A new detection tool can also be implemented to monitor packet traffic. The detection tool runs in the kernel state, and the traced data is processed and sent directly up to the user state.

II. RELATED WORK

A. the eBPF technigh introduction

eBPF[1], known as extended Berkeley Packet Filter, is a technology embedded in the Linux kernel that allows sandboxed programs to run in kernel space (e.g. ring-0). It can be used to enhance and extend kernel functionality without loading any additional kernel modules and without recompiling the kernel in a safe and secure manner.

The eBPF programs can run on hook points in the Linux kernel, with functions covering XDP, TC, Probe, Sockets, etc. Each function point can implement kernel-state tampering behavior by triggering special events

B. Service network research introduction

As mentioned above, service mesh technology emerged from the development of microservices. Figure 1 shows the architecture of Istio [1], nowadays the mainstream service mesh open source project.

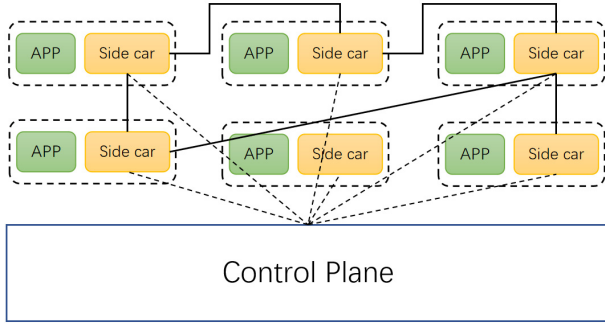


Figure1. Istio[1] service mesh architecture

Some academic research in cloud computing scenario has now used the idea of microservices and service mesh. [3]Sedghpour and Townsend detailed the current situation of service mesh and the future direction of development. [4] Kang and Shin introduced the possible security problems of the mainstream service mesh technology and proposed a solution for the new architecture. In [5] Song and Liu pointed out the importance of fault tracing and locating capability in complex service mesh architectures. An approach to optimize the traceability of microservices is proposed in the paper. [6] Used a novel service mesh control plane architecture to improve the efficient and economical deployment of service meshes on edge clouds.

Although the application of service mesh in cloud computing is expected to be studied in the future, few studies on the technology are currently reported.

Existing work has optimized the service mesh, making it usability-enhancing. However, none of these studies have considered how to effectively reduce latency and lower system overhead in complex microservice network environments as well as enhancements to service mesh scenarios in terms of operations and maintenance. To fill these gaps, this paper optimizes the current Service mesh architecture with eBPF techniques to achieve enhancements in network latency and observability.

III. EXPERIMENTAL ENVIRONMENT

A. Experimental Topology

The experimental topology is shown in Figure 2, where we use a total of three physical nodes to simulate a cloud environment and deploy a Kubernetes cluster. From this, we create a basic environment to meet the deployment of service mesh. On top of this environment, we deploy two sets of service mesh services, the first set of service mesh services with conventional architecture and the second set of service mesh services optimized by us.

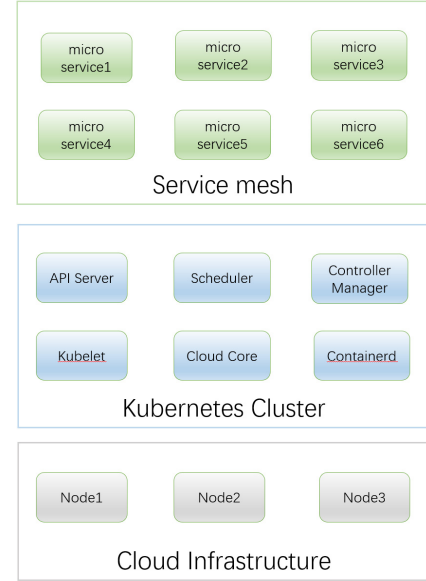


Figure2. Experimental Topology

B. Hardware Setup

The experimental environment for this paper uses three DELL R740 servers, each with the following main configurations

- 2 x Intel Xeon Silver 4214R 2.4GHz
- 256GB of RAM
- 2 x 4TB hard disk space
- 2 x 10Gbps NIC

In this article, Huawei CloudEngine S5731-H optoelectronic hybrid switch is used to implement network communication between three servers, and the configuration of the switch is

- 282/480Mpps packet forwarding rate
- 758Gbps/7.58Tbps switching capacity
- 20 Gigabit optical SFPs, 4 optical 10 Gigabit SFP+, 4 10 Gigabit SFP+

C. Software Setting

In this paper, Ubuntu 18.04 operating system with kernel version 5.4.0-65-generic was installed on three servers. the GTP5G module was installed on top of the operating system kernel to support the forwarding of GTP-U protocol data. The software configuration on top of the operating system is shown in Table 1.

Table 1. Service mesh software running environment

Environment configuration items	Configuration information
Compiler environment	● gcc v7.3.0 ● Golang v1.14.4
Database	linux/amd64 MongoDB v4.4
Software dependencies	git, cmake, autoconf, libtool, pkg-config, libmnl-dev, libyaml-dev
Network configuration	● enables ipv4 forwarding ● enable DNAT mode for data networks ● Disable system firewall
Container orchestration management system	Kubernetes v1.25.0
Container image repository	Harbor v2.6.0

IV. SOLUTION DESIGN AND IMPLEMENTATION

In this section, we analyze the service mesh according to the challenges encountered in edge computing scenarios and propose feasible novel architectures.

In distributed architectures, control and logic are usually designed separately. The same idea is used in the service mesh architecture: the control module implements functions such as monitoring, logging, flow limiting, fusing, service registration, protocol adaptation conversion, etc. The service module implements the business logic functions. The control module mentioned above is called a side car in the traditional service mesh architecture.

While the sidecar model has great advantages over reconfiguring applications, it runs as a container in the host and does not provide perfect isolation between the application and the data plane. This leads to these limitations as described below.

Intrusiveness: The sidecar proxy must be "injected" into the application by modifying the application's Kubernetes pod spec, and requires redirecting traffic from the application in the pod to sidecar. therefore installing or upgrading sidecar requires restarting the application pod, which can be disruptive to workloads. This can be disruptive to the workload.

Underutilization of resources: Since each sidecar agent is only used for the relevant workload in its pod, the CPU and memory resources of the sidecar must be conservatively allocated for each pod's worst-case scenario. This results in significant resource reservation, which can lead to underutilization of resources across the cluster.

Traffic disruption: Traffic capture and HTTP processing are typically performed by sidecar, and these operations are computationally expensive and may break applications whose

implementations are not fully compatible with the HTTP protocol.

High latency: Communication between sidecars and applications, and between sidecars and sidecars, also increases the time cost of microservice communication, making the path of packets through the kernel stack longer.

We design a modification of the host's kernel network stack to remove the service mesh architecture's dependence on the sidecar proxy. To achieve this, we designed a "kernel sidecar" based on the eBPF technology implementation. This design retains the advantages of being non-intrusive to the application, free from the language and technical constraints of the application service, and separating control and logic. It also takes advantage of the fact that eBPF runs in the kernel to reduce overhead and shorten the path of packets through the kernel stack.

A. Side Car

The most important function of the side car is to control the traffic coming out of or into the individual microservices. [5] Cilium was the first to propose a kernel network stack optimization scheme. The design of this paper refers to Cilium and uses eBPF technology in the kernel to achieve the effect of replacing the traditional side car by processing, hijacking, and forwarding capabilities of packets.

Specifically, the metrics that this part wants to optimize are the path length of packets in the host network stack, and the CPU and memory resources occupied by the side car agent.

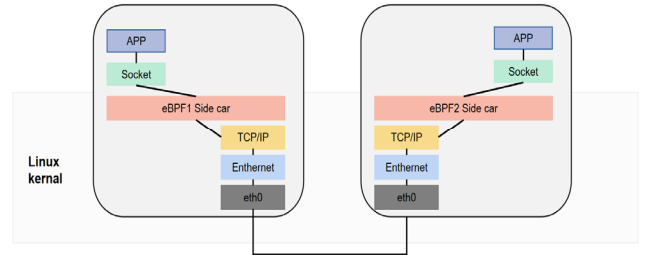


Figure3. Optimized service mesh architecture

Hijacking of traffic is done mainly through the eBPF program hook bind system call. eBPF program hijacks the bind system call and modifies the address. The hijacked traffic is forwarded to the eBPF program running in the kernel for processing.

Unlike iptables, which can set rules for each netns individually, the eBPF program attaches to the specified hook point and it takes effect for the whole system. In Kubernetes, except for the case of host network, each Pod has its own netns, and each netns has a unique cookie, so we save the corresponding netns cookie of the Pod that needs to use eBPF to complete traffic hijacking in the cookie_map. The eBPF program decides whether to modify the bind address by determining whether the netns cookie of the current socket is in the cookie_map.

As shown above, the hijacked traffic will be forwarded to the eBPF1 program. eBPF1 replaces the capability of the side car proxy in the traditional scheme. eBPF1 will use mTLS

encryption to authenticate, identify and forward the traffic for security purposes. The new architecture bypasses the tcp/ip layer and the protocol processing steps at the lower layers. This makes the path of traffic through the kernel protocol stack shorter and reduces the processing time of traffic in the kernel. Also as the eBPF program in the kernel replaces the huge number of edge carriage agents in the user state. This frees up memory and CPU resources that would otherwise be consumed by the large number of edge carriage agents.

The new edge-vehicle-less service mesh architecture, featuring smaller resource overhead and faster latency, makes it possible to deploy service mesh services in edge clouds or cloud computing nodes with fewer computational resources, driving larger-scale applications of service mesh in the future.

B. Observability and tracking

We are designed to examine the host's network stack to gather metrics on the performance of each connection. As well as event tracing for some problematic links. The metrics enable fast identification of bad performance and the traces enable fine-grained visibility and fast debugging.

Trace collection module: The trace module needs to be deployed in different layers of the network kernel stack. Calls to kernel functions for packet operations trigger the eBPF program of the trace function to obtain information about the packet and record it. In more design details, we plan different dimensions of the packet, such as IP quintet, layer name, timestamp, process name, thread name, CPU used to process the traffic, etc. Selectively collecting and processing a small portion of packets based on different dimensions can greatly reduce storage and memory overhead. It can effectively increase the efficiency when the application traffic is at its peak.

Data processing module: Draws the traffic topology of the service mesh based on the information processed by different modules. To visually restore the behavior and characteristics between microservices. See the traffic calls between microservices.

V. Conclusions

In this paper, we propose and implement an optimization scheme based on the mainstream service mesh architecture. It addresses the problems of resource consumption and high

latency in complex network environments caused by the need to deploy a large number of side-car agent containers in traditional architectures. By sinking the edge-vehicle agent to the host kernel state, it avoids frequent switching between user and kernel states and uses eBPF technology to perform customized operations in specific functions of the kernel to bypass the traffic and shorten the path in the kernel. The above issues are optimized while keeping the original functional logic unchanged. Also in terms of observable lines and traces. The use of eBPF technology to trace finer-grained network packet information solves to some extent the problems of observability and fast debugging of complex network environments for microservices. The new service mesh architecture proposed in this paper improves the resource utilization and availability of microservice applications in various scenarios, reduces the usage threshold and usage cost, and provides a basis for microservice promotion and application. Especially in the edge scenarios with fewer resources, the availability and ease of use of the service mesh are greatly improved.

In addition, further deployment and verification are needed in future work in terms of network security and reliability.

REFERENCES

- [1] Istio. <https://istio.io/>
- [2] eBPF. <https://ebpf.io/>
- [3] M. R. S. Sedghpour and P. Townend, "Service Mesh and eBPF-Powered Microservices: A Survey and Future Directions," 2022 IEEE International Conference on Service-Oriented System Engineering (SOSE), Newark, CA, USA, 2022, pp. 176-184, doi: 10.1109/SOSE55356.2022.00027.
- [4] M. Kang, J. -S. Shin and J. Kim, "Protected Coordination of Service Mesh for Container-Based 3-Tier Service Traffic," 2019 International Conference on Information Networking (ICOIN), Kuala Lumpur, Malaysia, 2019, pp. 427-429, doi: 10.1109/ICOIN.2019.8718120.
- [5] M. Song, Q. Liu and H. E., "A Micro-Service Tracing System Based on Istio and Kubernetes," 2019 IEEE 10th International Conference on Software Engineering and Service Science (ICSESS), Beijing, China, 2019, pp. 613-616, doi: 10.1109/ICSESS47205.2019.9040783.
- [6] Y. Li, D. Zeng, L. Chen, L. Gu, W. Ma and F. Gao, "Cost Efficient Service Mesh Controller Placement for Edge Native Computing," GLOBECOM 2022 - 2022 IEEE Global Communications Conference, Rio de Janeiro, Brazil, 2022, pp. 1368-1372, doi: 10.1109/GLOBECOM48099.2022.10000996.
- [7] GitHub - cilium/cilium: eBPF-based Networking, Security, and Observability